

Jumping Into the Frame

Reconstructing Interactive, Physically-Grounded 3D Environments
from a Single Image

Joshua Ford

Advisor: Marie-Paule Cani

August 23, 2026

Abstract

Single-image 3D scene generation has advanced rapidly, but most systems stop at a viewable, inert result: a panorama, a layered mesh, or a radiance field that supports novel-view synthesis but has no motion and nothing to interact with. We present **Into Frame**, a system that turns one photograph into an *inhabitable* environment, in which discrete objects exist as individual simulated assets, ground cover and vegetation respond to contact and wind, and the scene is lit by a physically based estimate of the original illumination, all at real-time VR frame rates. Our central design commitment is to reconstruct each part of the scene by a method suited to its physical role, rather than extracting everything from one representation: terrain, objects, ground cover, water and sky are handled separately and by different means. We outline the pipeline built on this principle and focus on four contributions: a projection from calibrated panoramic depth to a top-down height field that preserves each cell’s observed panorama pixel; a terrain reconstruction that keeps visual similarity with the original image while applying fluvial erosion to give plausible relief where nothing was observed; a pattern-texturing scheme that avoids the projective smearing an equirectangular photograph produces on the ground; and a point-process population synthesis that scatters vegetation and objects to match the spatial statistics of the real detections. We evaluate the system on several real-world images and report both what it does well and a set of failure modes we were able to measure precisely. We argue that the most transferable result is methodological: at this scale, most defects are not model failures but *assumption* failures, where a step valid for one scene category is silently applied to another.

1 Introduction

1.1 Motivation

A photograph is a two-dimensional projection of a three-dimensional world, and recovering that world from a single view is underconstrained: infinitely many scenes project to the same image. Humans nonetheless read a photograph as a place. We infer that the ground continues past the frame, that the occluded side of a rock exists, that the flowers in the foreground are individual plants rather than a printed texture. Building an environment that supports those inferences, one a person can *jump into*, is the problem this project begins to address.

Photographs are among the most abundant records of place we have: personal archives, cultural heritage documentation, scientific field imagery, and now, with generative models, images of places that never existed at all. A system that converts such an image into an environment a person can walk into changes what those archives are able to express. It also lowers the cost of 3D content authoring by several orders of magnitude relative to manual modeling or multi-view capture, and because the process is automatic, it puts that authoring within reach of a non-expert.

1.2 The Challenge

The difficulty is not one hard sub-problem but the interaction of several, each of which degrades the others:

Extreme ill-posedness.

A single view fixes neither absolute scale nor the geometry of anything it does not see. Roughly 5/6 of a full 360° panorama generated from a standard photograph is synthesized rather than observed, and everything behind every foreground object is unknown. Much of the world must be inferred.

Compounding error.

A pipeline that estimates depth, then geometry, then materials, then objects inherits every earlier error, and a discrepancy that is barely visible at one stage can be structural by the next.

Heterogeneous content.

The representation that suits a mountainside (a height field) is wrong for a tree (a mesh instance), which is wrong for a meadow (a scattered population), which is wrong for the sea (an animated surface). These representations must nonetheless coexist in one scene: they need to share a coordinate frame and a scale, agree on how they are lit, and not seam visibly where they meet.

Interactivity constraints.

The output must render stereoscopically at headset frame rates. That budget constrains every rendering choice, and it binds hardest on the animated, simulated elements that make the scene interactive in the first place.

1.3 Goals

Our goal is a system that takes one image and produces an environment which is *interactive* (objects animate and interact with their surroundings), *decomposed* (discrete objects exist as individual entities, not painted texture), *plausible* beyond the observed region, and *performant* at VR frame rates.

2 Related Work

2.1 Single-image panorama generation

Extending a narrow-field photograph to a full 360° environment is the first step in most single-image scene systems. Diffusion models adapted to panoramic geometry are the dominant approach. CubeDiff [18] repurposes a perspective latent diffusion model for panorama generation by operating on cubemap faces with synchronized attention. DreamCube [12, 11] extends this to RGB-D panoramas through multi-plane synchronization, producing geometry alongside appearance. PanoDreamer [30] takes an optimization-based route to coherent single-image 360° scenes, and LayerPano3D [41] decomposes the panorama into depth layers to support larger viewpoint changes. Our system supports several such backends, defaulting to a FLUX-based [5] outpainting path, with WorldGen [40] and DreamCube as alternatives. The relevant distinction is that we treat the panorama as an intermediate representation rather than as the deliverable.

2.2 Single-image 3D scene generation

A growing body of work targets navigable 3D scenes from one image. SphericalDreamer [34] generates navigable immersive worlds; SonoWorld [17] extends single-image scene generation to audio-visual output; 360Anything [37] lifts images and videos to 360° geometry-free representations;

and WorldGen [40] produces complete 3D scenes rapidly. These systems generally optimize for visual fidelity from viewpoints near the original camera. Into Frame differs in prioritizing *physical decomposition*. Our output is not a single geometry proxy but a set of typed entities with distinct behavior: a collidable terrain mesh, individually placed object meshes, an instanced ground-cover population, and an animated water surface. Each of these is required before a scene can be dynamic rather than merely viewable.

2.3 Depth estimation and panoramic depth

Monocular depth estimation underpins all geometry in the pipeline. We use Depth Anything 3 [24] for perspective depth and Depth Any Panoramas [25] for equirectangular depth, the latter avoiding the distortion artifacts that arise when a perspective model is applied to equirectangular imagery. A recurring difficulty, which we address explicitly in section 4, is that monocular depth is relative and saturates: real distant terrain and sky receive indistinguishable predictions, so calibrated depth at a ridge crest is an extrapolation rather than a measurement.

2.4 Terrain modeling and heightmap synthesis

Recovering terrain from imagery connects to a long line of graphics work. Pixels2Peaks [16] converts terrain images to heightmaps directly. Terrain modeling from feature primitives [6] constructs terrain from sparse user-specified features, and Earthbender [1] offers interactive stylistic heightmap generation. WorldBrush [4] synthesizes procedural virtual worlds from examples, including the distribution of scene elements. For physical plausibility of unobserved relief we use Landlab [2, 8, 15], an Earth-surface dynamics toolkit, to apply fluvial incision so that invented terrain obeys drainage structure rather than reading as smooth interpolation or undirected noise. The regions we never observed are therefore shaped by the same process that shapes real landscapes.

2.5 Texture synthesis

Projecting a panorama onto ground fails at grazing incidence: a single panorama row covers meters of ground at distance. Pattern-based texturing [28] addresses exactly this class of problem by assembling surfaces from patterns rather than resampling a projection, and our terrain texturing follows that idea using patches extracted from the real photograph. Where synthetic material is unavoidable we use FLUX [5] and latent diffusion models [33], with Swin2SR [3] for super-resolution.

2.6 Segmentation, detection and recognition

Object decomposition uses SAM 2 [32] for segmentation and video tracking, Grounding DINO [26] for open-vocabulary detection and label verification, CLIP [31] for embedding-based classification, BLIP [20] for captioning, Recognize Anything [43, 10] and its open-set successor RAM++ [9] for scene tagging, SegFormer [39] for semantic region typing, and BiRefNet [45] for high-resolution matting. Each of these models solves a similar but subtly different problem. We combine them rather than choosing between them, using their agreement where they overlap to build a more reliable reading of the scene than any one model provides alone.

2.7 Object reconstruction and inpainting

Individual objects are reconstructed with SAM 3D [36], with SPAR3D [13] and TRELLIS [38] as alternatives. Removing foreground objects so the ground behind them can be reconstructed uses inpainting: LaMa [35], MAT [21], Inpaint Anything [42], and ObjectClear [44], the last targeting removal of both objects and their effects.

2.8 Lighting

Relighting reconstructed content so that it matches the original photograph requires recovering the scene illumination. We use LuxDiT [22] for HDR lighting estimation and IntrinsicDiffusion [27] to separate albedo from shading, so that assets carry material color rather than baked-in sunlight and can be lit by our own lighting algorithm.

3 Overview

3.1 Design principle: role-specialized decomposition

The organizing commitment of Into Frame is that a scene is not one thing. Ground, discrete objects, ground cover, water and sky differ in how they must be reconstructed, how they are represented, how they are textured, and how they behave at runtime. Forcing them through a single representation means either accepting the limitations of that representation or developing a more general one, and the latter was outside the scope of this project.

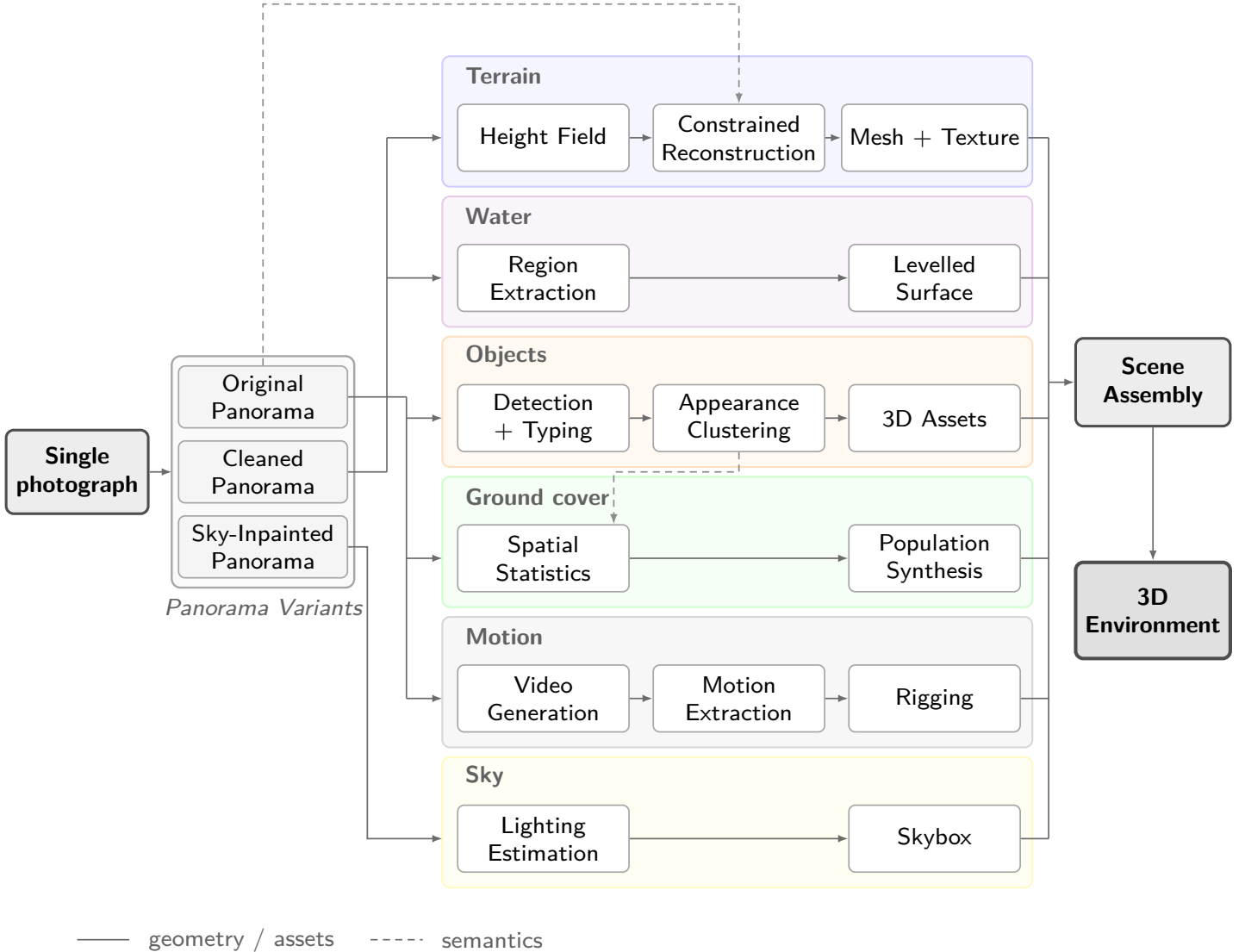


Figure 1: The pipeline decomposes a single image into six tracks with different physical roles. Solid arrows carry geometry, dashed arrows carry semantics. The three panorama variants at the left are maintained deliberately: geometry is derived from the cleaned panorama, semantics from the original.

Figure 1 shows the resulting structure. The pipeline begins by building three panoramic representations of the scene, each of which serves a different consumer downstream:

Original Panorama.

A synthetic equirectangular panorama generated from the input photograph by WorldGen [40], with FLUX [5] outpainting, CubeDiff [18] and DreamCube [12] available as alternative backends. This is the baseline scene representation, and the one every semantic stage reads from, because it is the only variant in which the real objects are still present.

Cleaned Panorama.

The original panorama processed with ObjectClear [44] to remove foreground objects, followed by a FLUX LoRA correction pass that restores equirectangular consistency at the seams. This gives an unobstructed view of the environment, better suited to terrain reconstruction, where occluders would otherwise punch holes in the ground.

Sky-Inpainted Panorama.

The region above the horizon, completed with FLUX [5] inpainting where objects were removed, producing a seamless sky used as both skybox and image-based lighting source.

These three panoramas are then taken in concert to generate the rest of the scene:

Terrain.

A single continuous height field, meshed with variable density, carrying collision. Reconstructed from calibrated panoramic depth and completed by a terrain constraint solver.

Water.

Regions typed as water, extracted as a separate leveled surface with an animated shader.

Objects.

Discrete entities detected, classified, clustered by visual similarity, and reconstructed across the scene.

Ground cover.

Populations too numerous to place individually as reconstructions, scattered by density over a mask derived from the semantics and color of the panorama.

Motion.

Per-object animation, guided by a generated video of the scene and used to decide which categories are rigid bodies and which need rigging.

Sky.

The panorama above the horizon, used as both skybox and image-based lighting source.

3.2 Stage sequence

The pipeline defines 42 stages, of which 38 are active in the default configuration. Table 1 groups them by track and section 10 reports what each costs to run; table 2 gives the model behind each stage that uses one. Stage numbers follow configuration order throughout. The gap at 36–37 in table 1 is two stages that are defined but disabled by default, and so belong to no track.

The panorama stages build a 360° representation of the scene. Semantics then runs against that representation to identify the objects within it. Cleanup produces a second panorama with those objects removed, better suited to terrain generation, and the geometry stages recover depth and a metric spatial understanding of the scene from it. Terrain is meshed and eroded using established techniques. Objects are detected and their spatial distributions learned, after which the scene is assembled from those statistics. Finally, a video generation model guides animation for each object and determines whether it should be treated as a rigid body.

Table 1: Pipeline stages grouped by reconstruction track.

Track	Stages	Purpose
Panorama	1–4	Caption, Perspective Depth, 360° Outpainting, Super-Resolution
Semantics	5–8	Panoramic Depth, Object Segmentation, Classification, Typing
Cleanup	9–11	Foreground Removal, Equirectangular LoRA Correction
Regions	12–15	Semantic Region Typing (original and cleaned), Lighting Estimation, Sky Inpainting
Geometry	16–19	Panoramic Depth, Metric Calibration, Height Field, Region Map
Terrain	20–25	Constrained Reconstruction, Erosion Refinement, Meshing, Texturing
Objects	26–30	Recognition, Detection, Instance Refinement, Correlation, Clustering
Population	31–33	Spatial Statistics, Population Synthesis, Ground Cover
Assembly	34–35	Asset Generation, Scene Assembly and Placement
Animation	38–42	Video Generation, Motion Extraction, Rigging, Animation

4 The Panoramic Data

Every track in fig. 1 consumes the same three artifacts: a 360° panorama, an aligned metric depth map, and a per-pixel semantic typing. The depth map and semantic typing are both derived from the generated panorama, which extends the provided input image into a complete 360° view of the scene.

4.1 Panorama Generation and Metric Calibration

The input image is first processed by generating a synthetic caption and a perspective depth estimate, then outpainting the image to a full equirectangular panorama, which is super-sampled to its working resolution. Panoramic depth is estimated on the result [25].

The depth model in *Depth Any Panoramas* is specifically built to generate accurate depth for equirectangular panoramic images. However, the depth values it provides are relative: it does not produce real, metric depth. We therefore calibrate the panoramic depth output against a second estimate that does carry metric meaning.

To achieve this we run Depth Anything 3 [24] against the original input image, which gives a metric baseline over the region the photograph actually observed. The two maps are related by fitting a monotonic response curve rather than a single scale factor, because the relationship between the two models’ outputs is not linear.

Fitting the response curve. Let $r \in [0, 1]$ be the panoramic model’s raw prediction and d the metric depth from the perspective model. We project the hero photograph into the panorama using the same field of view that was used to place it during outpainting, which gives a set of corresponding samples $\{(r_i, d_i)\}$ over the overlap region. Sky pixels and non-finite or non-positive samples are discarded. The surviving samples are partitioned into $N = 14$ quantile bins of r , and each bin contributes one knot

$$c_b = \text{median}\{r_i : i \in \mathcal{B}_b\}, \quad m_b = \text{median}\{d_i : i \in \mathcal{B}_b\}, \quad (1)$$

where $\mathcal{B}_b$ is the set of samples in bin b . Taking medians rather than means keeps a handful of

Table 2: Model behind each pipeline stage that depends on one, in configuration order. Stages that use no model are omitted, so the numbering is not contiguous; table 1 gives the full sequence. When a stage supports several backends, the default is listed first. Stages marked † are defined but disabled in the default configuration.

#	Stage	Model
1	Captioning	BLIP [20]
2	Depth Generation	Depth Anything 3 [24]
3	Panorama	WorldGen [40]; or FLUX [5], CubeDiff [18], DreamCube [12]
4	Panorama Supersampling	Swin2SR [3]
5	Panorama Object Depth	Depth Any Panoramas [25]
6	Object Segmentation	SAM 2 [32], with BiRefNet [45] matting
7	Object Classification	BLIP [20] captioning with keyword matching
8	Object Typing	CLIP [31], with Grounding DINO [26] label verification
9	Panorama Inpainting†	FLUX [5]; or LaMa [35]
10	Panorama Foreground Inpainting	ObjectClear [44]
11	Panorama LoRA Correction	FLUX [5] with an equirectangular LoRA
12	Panorama Regions	SegFormer [39]
13	Panorama Regions (Terrain)	SegFormer [39]
14	Panorama Lighting	LuxDiT [22], with IntrinsicDiffusion [27] albedo/shading separation
15	Skybox Inpainting	FLUX [5]
16	Panorama Depth	Depth Any Panoramas [25]
22	Terrain Noise Refinement	Landlab [2, 8, 15] fluvial incision
25	Terrain Texture Generation	Pattern-based texturing [28], with LaMa [35] completion; or FLUX [5]
26	Object Recognition	RAM++ [9]
27	Object Detection	Grounding DINO [26]
28	Object Instance Refinement	SAM 2 [32]
29	Object Correlation	SAM 2 [32]
30	Object Category Clustering	DINOv2 [29], with CLIP [31] corroboration
33	Grass Cover	SAM 3D [36] for the tuft mesh
34	Panorama Asset Generation	SAM 3D [36]; or SPAR3D [13], TRELLIS [38]
36	Foreground Inpainting†	LaMa [35]
38	Video Generation	LTX-2 [23]
39	Video Object Extraction	SAM 2 [32]

mismatched samples, typically foreground objects that survive into the overlap, from dragging a knot. Depth must not decrease as the raw prediction increases, so the knots are made monotonic by a running maximum, $\tilde{m}_b = \max_{k \leq b} m_k$, which prevents a noisy bin from punching a dip into the curve. Calibration is attempted only when at least 300 valid overlap samples survive; below that the photograph barely overlaps the panorama, typically an extreme field of view or a mostly-sky shot, and the panorama depth is left uncalibrated rather than fitted unreliably.

Metric depth is then recovered by interpolating $(c_b, \tilde{m}_b)$. The correspondence is treated as a property of the depth model rather than of the particular image, so the same fitted curve is applied to every panoramic depth map in the scene, which is what keeps the two depth maps that must agree on a common scale.

Extrapolating beyond the fitted range. The overlap region only ever covers the photograph’s own near field, a few to a few tens of meters, so it never contains genuinely distant terrain. Interpolation alone would flat-clamp every raw value past the fitted range onto the curve’s last knot, collapsing a mountain a kilometer away and a tree just past the fitted range onto the same depth. We instead extrapolate in log-depth space, since the raw prediction approaches its ceiling asymptotically as true depth grows,

$$d(r) = \exp(\log \tilde{m}_N + s(r - c_N)), \quad s = \frac{\log \tilde{m}_N - \log \tilde{m}_{N-k}}{c_N - c_{N-k}}, \quad (2)$$

with the slope s estimated from the last $k = 3$ knots, and symmetrically at the near end. This is capped at $d \leq \lambda \tilde{m}_N$ with $\lambda = 20$. The cap matters because the raw output saturates well before true distance does: a mountainside a few hundred meters out and a hard-clamped sky pixel both read at essentially the same near-unit ceiling. A slope estimated from a few tail bins and then exponentiated over that gap is extremely sensitive to noise, and in practice produced fabricated depths of tens of kilometers for every pixel sharing a saturated raw value. A single outlier of that size dominates anything downstream that treats panorama depth as a trustworthy scale, which silently discards real terrain as out of bounds rather than merely far. The cap cannot recover a distance the model genuinely cannot resolve, since there is no signal left to extrapolate from once the prediction saturates, but it keeps far content bounded and ordered.

Fitting the scene into the terrain budget. The calibrated map must finally fit inside the terrain grid’s own footprint, $D_{\max} = 100$ m, which is half the 200 m grid extent; nothing farther can land inside that grid in any case. The obvious approach, a uniform rescale by $D_{\max}$ over the farthest non-sky pixel, preserves relative distances exactly and is the wrong invariant to protect. The scale would then be set by the single most distant thing in the scene, which for a landscape photograph is effectively unbounded, and it silently invalidates the pipeline’s second and more load-bearing metric anchor: the assumed camera height, which the height map relies on to place ground at $-h$ directly below the camera. On one capture of Mount Rainier, fitting a summit kilometers away into 100 m divided the whole map by roughly $6.5\times$; ground 1.37 m below the camera came back as 0.21 m of depth, and the reconstructed surface ended up around 0.9 m *above* eye level with the camera buried inside its own terrain. A flat clamp is wrong for the opposite reason, collapsing a 120 m ridge and a 900 m peak onto one wall.

We instead compress only the far end, leaving the near field metrically intact. With a knee at $k = 0.6 D_{\max}$ and tail $t = D_{\max} - k$,

$$d' = \begin{cases} d, & d \leq k, \\ k + t \left(1 - \exp\left(-\frac{d-k}{t}\right) \right), & d > k. \end{cases} \quad (3)$$

This is continuous and C^1 at the knee, since both branches have unit slope there, strictly monotonic everywhere, and asymptotic to $D_{\max}$, so no pixel can exceed the budget without a

hard clamp. Distant content keeps its ordering and stays distinguishable; it simply stops being proportional, which it never meaningfully was once a mountain kilometers away had to fit inside a 200 m grid. Because the curve is fixed by $(D_{\max}, k)$ alone rather than by the data’s own maximum, there is no pool of samples for saturated sky to contaminate, and applying the same deterministic function to both depth maps places the same real-world point at the same distance in each, with no scale factor to thread between them.

Consequences. The result is metrically compressed. Distances to close objects are accurate, while distant objects are systematically shrunk. Since we assume the user explores the scene only on the scale of a few meters, the visual effect of this is minimal: the scene relies on forced perspective, and a large mountain far away reads much like a smaller one closer in. The compression is not free, however, and section 11 returns to a defect it causes, where object size is computed as angular extent times depth and therefore inherits the same shrinkage.

4.2 Object-Removed Panorama

To assist with terrain generation we produce a second panorama with foreground objects removed, using ObjectClear [44]. That model is trained to remove objects together with their effects, such as reflections and cast shadows, rather than only the objects themselves. We select the objects to remove by thresholding on depth, taking those closest to the camera, and run the model across the foreground region of the panorama.

The result is a clean panorama better suited to terrain reconstruction. Taking depth from the panorama with objects still in it instead produces spurious depth values and raised bumps whenever an object stands in front of the ground, since the surface behind it was never observed; removing the objects first gives the depth model an unobstructed view of the terrain.

4.3 Semantic Region Map

A semantic region map is produced by determining per-pixel region types (sky, water, terrain, ground, vegetation, built objects, road, trail) from the panorama. This map drives much of the downstream scene understanding. We use SegFormer [39] to determine these semantic regions, which are then used for terrain construction and for object understanding and placement.

5 Terrain Geometry

5.1 Projection with UV retention

The height field is a 4096^2 top-down grid spanning 200 m. Each panorama pixel below the horizon is unprojected to a 3D point using calibrated depth and accumulated into the grid cell containing its (x, z) .

The contribution here is what is retained alongside the height. For every cell that receives a genuine measurement we store the panorama (u, v) it was observed through, together with a mask distinguishing directly measured cells from those later filled by interpolation. This costs two float channels and removes an entire class of error.

The alternative — re-deriving (u, v) from the final geometry — is what a naive implementation does, and it fails in a specific way. The elevation angle of a ground point is $\varphi = \arcsin(y/d)$, so near the nadir a small error in y produces a large error in φ and therefore in panorama row. Since y has by then passed through reconstruction, erosion and meshing, it is not the value that was measured. Retaining the observed UV means downstream texturing samples the correspondence that was actually observed.

5.2 Constrained harmonic completion

The projected height field is sparse and uneven: dense near the camera, sparse at range, and empty wherever an object occluded the ground. Completion is posed as a constrained boundary-value problem. Cells with direct, high-certainty measurements become Dirichlet boundary conditions; distant ridge anchors extracted from the sky silhouette are added as further fixed values; the remainder is solved as a discrete harmonic problem

$$\nabla^2 h = 0 \quad \text{on } \Omega_{\text{unknown}}, \quad h = h_{\text{obs}} \quad \text{on } \partial\Omega_{\text{obs}}, \quad (4)$$

using a sparse linear solve. Measured terrain is therefore never modified, and unobserved terrain is the smoothest surface consistent with it.

5.3 Ridge anchoring and its assumption

Terrain beyond depth range has no usable measurement, so its elevation comes from the sky boundary: for each panorama column, the first non-sky pixel below sky gives an elevation angle, placed at a fixed radius on the grid. This is a deliberate scale compromise — a mountain kilometres away cannot be walkable terrain in a 200 m grid, so it is brought close and shrunk, preserving silhouette shape rather than true distance.

The assumption embedded here is that the sky boundary is the crest of *ground*. Section 11 shows what happens when it is not.

5.4 De-spoking

The equirectangular-to-grid projection maps each panorama column onto a radial line on the grid. Any per-column bias in the depth prediction therefore becomes a radial streak, and because those streaks lie in the *observed* data — around 89% of solve nodes are fixed — the harmonic solve cannot remove them.

We remove them with a targeted filter that subtracts only the tangential high-frequency component, computed as a polar round-trip difference

$$h' = h - (P(h) - G_\sigma^\theta(P(h))), \quad (5)$$

where P is the polar resample and G_σ^θ an angular Gaussian blur. Taking the difference cancels the resampling error, which would otherwise bake in concentric rings: resampling error is around 1.2 m against a spoke signal of roughly 0.2 m. Validated end-to-end, the radial elevation profile is unchanged (18.1 m/30.7 m/21.8 m at $r = 45/60/80$ m against 18.1 m/30.7 m/21.9 m before), the cliff mask is bit-identical, and the mean absolute change is 0.19 m concentrated in the spoke regions.

5.5 Erosion refinement

Harmonic completion is smooth by construction, which is correct as an estimator and wrong as terrain. Real landforms carry drainage structure. We therefore run fluvial incision over the completed field using Landlab [2, 8], restricted to unobserved cells so measured ground is preserved. Erosion strength is modulated by a jaggedness map derived from the ridge silhouette, so the invented relief matches the character of the visible terrain.

5.6 Meshing

The height field is triangulated by Poisson-disc sampling with a near-camera density bias, then Delaunay triangulation, giving natural level-of-detail without axis-aligned artefacts. Water regions are extracted as a separate surface, and the terrain beneath is depressed so an animated water plane never clips through its bed. Disconnected landmasses above a plausibility threshold are

extracted as separate formation meshes. A coarser collision mesh is generated by the same procedure with sparser parameters.

Each vertex carries a second UV channel holding the retained panorama coordinate from section 5.1. Because that channel is per-vertex and interpolated across triangles, triangles spanning an occlusion boundary — where grid-adjacent vertices observed very different panorama rows — smear a wide band of image across a small piece of ground. We detect these geometrically, comparing each triangle’s UV span against the span its own size and distance predict, and collapse the offending corner. Detection must be geometric rather than a fixed pixel budget, because a near triangle legitimately spans tens of degrees while the same triangle at range spans under two.

6 Terrain Appearance

6.1 Why projection alone fails

The obvious way to texture the ground is to project the panorama onto it. This fails in two distinct ways.

First, *grazing incidence*. Equirectangular rows compress with distance: at 80m a single panorama row covers roughly 12m of ground when the terrain is flat, against 0.5m when the horizon is anchored at its true elevation. The result is radial smearing.

Second, and less obviously, *upright content*. Projecting assumes the image at elevation φ is ground at radius $h/\tan(-\varphi)$. That holds for bare terrain and fails completely for anything vertical: a 0.5m flower spans a range of elevations, each landing at a different ground radius, so one flower smears into a streak metres long. Rendered top-down this produces a radial pinwheel. It is also double-counting, since the same flowers are placed as 3D instances on top of the ground.

6.2 Layered splat material

Texturing is therefore a weighted blend of layers. A panorama layer supplies real photographic colour where projection is trustworthy, gated by a visibility weight that falls off near the nadir and the horizon and is withheld where a texel’s own panorama pixel is vegetation or built structure rather than ground. Synthetic material layers, generated per region type, supply the rest.

6.3 Real-material pattern texturing

Where the panorama layer is withheld, we prefer real material over generated material. Following pattern-based texturing [28], small reference patches are extracted from the most solid interior of each region in the panorama and flattened — by fitting a local plane to the patch’s own depth points and reprojecting, which removes genuine oblique foreshortening rather than only equirectangular stretch — then assembled over the mesh’s own triangles with blended borders so the tiling does not read as faceted.

Reference patches are drawn from the object-removed panorama with real ground colour restored from the original wherever foreground removal ate the ground rather than an occluder. Where an occluder genuinely was removed, its footprint is refilled by inpainting from the surrounding real ground, so what remains is neither the occluder nor fabricated fill.

7 Object Decomposition

7.1 Detection, typing and verification

Objects are segmented on the original panorama with SAM 2 [32], tiled and merged to handle equirectangular distortion. Each crop is classified by CLIP against a fixed category set, with a BLIP caption [20] as fallback where the image pass is indeterminate. Labels are then verified

by asking Grounding DINO [9] to localise the proposed label in the crop: CLIP proposes, an independent model disposes.

The reason for verification is that CLIP scores a fixed label set and always returns a ranking, so on a capture where it has no signal that ranking is near-uniform noise with a winner. On one alpine capture, of 359 crops the 20 that survived CLIP’s own gates all scored within a 0.019–0.034 top-5 spread: junk crops and real flowers were numerically indistinguishable.

7.2 Correlation and clustering

Surviving detections are grouped by class, then sub-clustered by visual similarity using DINOv2 embeddings and agglomerative clustering, splitting a class into appearance variants (flower colours, tree species). Bucket counts are bounded, and undersized buckets folded into their nearest neighbour, because an unbounded distance cut produces one bucket per instance — measured at 302 groups on one capture, each demanding its own reconstruction.

7.3 Asset generation and level of detail

One mesh is reconstructed per (class, bucket) group with SAM 3D [36] and instanced at every member’s position. Two selection decisions matter. The crop fed to reconstruction is chosen by *resolution* among those passing quality gates, not by classifier score, because a single-view reconstructor given too few pixels does not return a worse object but a different one — a flat sheet. And the mesh-versus-billboard decision is made on *projected angular size* rather than distance, since distance only proxies resolvability when every object is the same size.

Reconstructions are validated against their detections before placement: a mesh too flat in any axis, or whose aspect ratio disagrees with its detection’s, is rejected in favour of a crossed-card or billboard representation.

8 Population Synthesis

A detector finds tens of flowers in a meadow that contains thousands. Placing only what was detected leaves a scene correct everywhere it was measured and empty everywhere it was not, which reads worse than either extreme — the eye sees the sampling pattern rather than the meadow.

Two mechanisms fill that gap, and they are deliberately separate because they answer different questions. Population synthesis asks *where else would instances of this class be, given where these ones are*. Ground cover asks *what is the ground made of*, a question with no discrete instances to generalise from at all.

8.1 Synthesis from observed statistics

For each (class, region-type) pair with enough real detections to constitute evidence, we measure the spatial statistics of those detections as a pair-correlation histogram, and synthesise additional instances matching them over the rest of the matching region. This is a point-process formulation [7], closely related to appearance-guided element arrangement [14] and in the spirit of example-based world synthesis [4].

Three properties of the implementation matter more than the choice of estimator.

Density is measured in world units, not image units. Instances per square metre is computed from the exemplar count over the exemplar hull’s area, a property of the distribution alone. An earlier version inferred the count per output tile from the ratio of candidate positions falling inside the input versus output boundary, which is only a proxy for areal density if both candidate grids are built at the same points-per-area rate. It is also unstable: three adjacent tiles

of the same group reported support values of 349, 4220 and 4220, and requested 290, 4 and 7 points respectively.

Density falls off radially, and the band is measured. Painting a learned density uniformly across a region spends the entire instance budget in the far field, where it buys nothing, and a fixed radius is wrong in the other direction for a class that lives in a band. The full-density radius is therefore extended to where the class was actually observed, and a near edge is imposed where nothing was observed closer. The asymmetry is deliberate: exemplar distance is bounded by *detectability*, not by where a class exists — a meadow is flowers to the horizon and only the nearest metre resolves into detections — so the rule only ever extends the outer radius, while the near edge is taken from a low percentile so a single mis-typed foreground crop cannot collapse it.

Synthesis is additive. Painted points are appended past the existing instance count and never displace a real detection. This keeps the measured content authoritative, but it does mean the two populations can collide, which section 11 returns to.

8.2 Ground cover

Grass is not synthesised from detections, because a detector does not usefully enumerate blades of grass. It is scattered by density over a mask, and the interesting part is how that mask is derived.

The mask comes from the region typing of the *original* panorama, for the reason given in section 4: the object-removed panorama has had the meadow erased along with the objects standing in it. But region type alone is too blunt in a way that matters here. The semantic label set folds **grass**, **earth** and **field** together with **sand**, **snow** and **ice** into one ground class, so a snowfield and a meadow are indistinguishable by type — and an alpine meadow is typically ringed by melting snowpack that types identically to the wildflowers beside it.

Colour settles it. Each candidate cell is tested against the excess-green of the panorama pixel it was observed through — available directly, because that pixel is the one retained in section 5.1. The threshold is deliberately a low bar rather than a test for lushness: dry grass sits near zero on this scale while sand runs about -0.12 and snow lower still, so it removes only surfaces that are definitively not vegetation. The same measurement is then reused as a *weight* rather than a veto, so density tracks how green the ground actually is instead of being uniform over whatever survives the threshold.

Tufts are placed as fixed-orientation crossed cards rather than camera-facing billboards. A billboard is a single quad that turns to face the viewer, which works for an upright subject at eye level and collapses to a line — then swings through the ground plane — for ground cover underfoot.

9 Lighting and Animation

The tracks described so far produce geometry and materials. Two things remain before the result reads as a place rather than a diorama: it has to be lit consistently, and it has to move.

9.1 Illumination and de-lighting

Scene illumination is estimated from the panorama with LuxDiT [22], which recovers an HDR environment and a dominant light direction. The client uses both: the environment as an image-based lighting source, and the direction as a directional light casting the scene’s shadows.

Estimating the light is only half the problem. Every asset in the scene is cut from a photograph, so it already carries the illumination of the moment it was taken — shadows, directional shading, and the colour of the light. Rendering that under a second, freshly estimated light doubles the illumination and the result reads as flat and over-exposed. Object crops are therefore de-lit with

IntrinsicDiffusion [27], which separates albedo from shading, so what the client relights is material colour rather than a photograph.

Two implementation details are worth recording because both are consequences of the pipeline’s structure rather than of the models.

First, de-lighting is applied only to the crops that will actually be rendered — the mesh representatives and the curated billboard pools — not to every detection. IntrinsicDiffusion is a diffusion model and the cost is per image; on a capture with several hundred detections the difference is a substantial fraction of the total run.

Second, the de-lit image is written under a *new* key rather than replacing the original crop. Video tracking (below) runs after this stage and matches crops against a video rendered from the original, fully-lit panorama, so handing the tracker an albedo image would make it match its own subject worse. Rendering wants albedo; tracking wants the photograph; both are kept.

9.2 Motion

Motion is recovered rather than authored. A short video is generated from the panorama, each detected object is tracked through it with SAM 2 [32], and the resulting trajectory is classified into a motion type. Category meshes for classes that should move are then rigged with a sway skeleton, and the client animates them.

Only classes that can plausibly move are tracked, and the per-class tracking budget is capped. Without a cap the cost is unbounded in a way that is easy to miss: a single class routinely splits into dozens of appearance buckets, so a per-bucket limit barely constrains anything — on one capture a “tree” class occupied 65 buckets, most holding one or two instances. Capping per class rather than per bucket is what makes the stage affordable. Synthesised population points are excluded entirely; they never appeared in the video, so there is nothing to track.

9.3 A shared wind field

Foliage sway, grass motion and the water surface are driven by one scene-global wind field rather than by independent per-object oscillators. The field is an ambient base strength plus discrete gust events, and each point’s gust is delayed by how long the wind takes to physically reach it — distance along the wind axis divided by gust speed.

That delay is the entire mechanism, and it is what distinguishes a scene that moves from one that merely wobbles. Independent oscillators produce uniform agitation everywhere, which reads as noise. A delayed pulse produces a gust that visibly begins at one side of the scene and sweeps across it, with grass, foliage and water all responding in step because all three sample the same field. The water shader evaluates the same gust formula in HLSL that the foliage sway evaluates in C#, so the swell rises with the same gust front that bends the grass.

10 Implementation

10.1 System architecture

The system is split into a Python generation server and a rendering client. The server executes the pipeline and exposes the result as a `.frame` archive over a WebSocket control channel and an HTTP asset channel, so a client streams assets as they become available rather than waiting for the full run.

Two clients exist. A Unity client (C#, URP) targets desktop VR and carries the runtime shading work — a terrain splat shader blending the layered material of section 6, an animated water surface, and wind-driven foliage sway sharing one global wind field so that grass, foliage and water gust together. A native macOS client (Swift, Metal 4) renders the same scene and streams it to an Apple Vision Pro through a remote immersive space.

Table 3: Learned components and their role in the pipeline.

Role	Model	Notes
Captioning	BLIP [20]	Scene and per-crop captions
Scene tagging	RAM++ [9, 43]	Open-set tags; used as a scene prior
Perspective depth	Depth Anything 3 [24]	Input image
Panoramic depth	DAP [25]	Equirectangular-native
Panorama synthesis	FLUX [5]	Alternatives: DreamCube [12], World-Gen [40]
Super-resolution	Swin2SR [3]	Panorama upscaling
Segmentation	SAM 2 [32]	Objects; also video tracking
Matting	BiRefNet [45]	High-resolution cutouts
Region typing	SegFormer [39]	ADE20K classes folded into 9 region types
Classification	CLIP	Fixed category set
Verification	Grounding DINO [9]	Independent label corroboration
Similarity	DINOv2	Appearance clustering into buckets
Object 3D	SAM 3D [36]	Alternatives: SPAR3D [13], TREL-LIS [38]
Inpainting	ObjectClear [44]	Alternatives: LaMa [35], MAT [21]
Lighting	LuxDiT [22]	HDR environment estimation
De-lighting	IntrinsicDiffusion [27]	Albedo extraction for assets
Erosion	Landlab [2]	Not learned; geomorphological simulation

Maintaining two renderers is a source of divergence worth recording: features added to one do not automatically exist in the other. The water shader described in section 6, for instance, exists only in the Unity client, so on the Metal client water falls back to the material parameters baked into the asset. Keeping the baked material in step with the shader defaults is therefore not cosmetic — it is what one of the two clients actually renders.

10.2 Models

Table 3 lists the learned components. The pipeline is deliberately model-agnostic where alternatives exist: panorama generation, object reconstruction and inpainting each have several interchangeable backends selected by configuration, which was essential during development because backend quality varied substantially by scene type.

10.3 Runtime and scene complexity

A full run on the Mount Rainier capture takes 93 min on a single GPU across 38 active stages. The distribution is heavily skewed: object segmentation (29.5 %) and per-group 3D reconstruction (25.2 %) together account for over half, with video generation and tracking (16.5 %) next. The entire terrain track — height field, reconstruction, erosion, meshing and texturing — is under 10 %, which is worth noting given it is where most of this report’s technical contribution lies. Optimising the pipeline means optimising the learned components, not the geometry.

The resulting scene contains a 79 982-triangle terrain mesh, a separate 20 228-triangle collision mesh, 29 shared category meshes totalling 4.0 MB, and 9475 placed entities of which the overwhelming majority are instanced ground cover. Because assets are shared per (class, appearance) group and instanced, entity count and asset payload are effectively decoupled — the property that makes a meadow of nine thousand plants affordable.

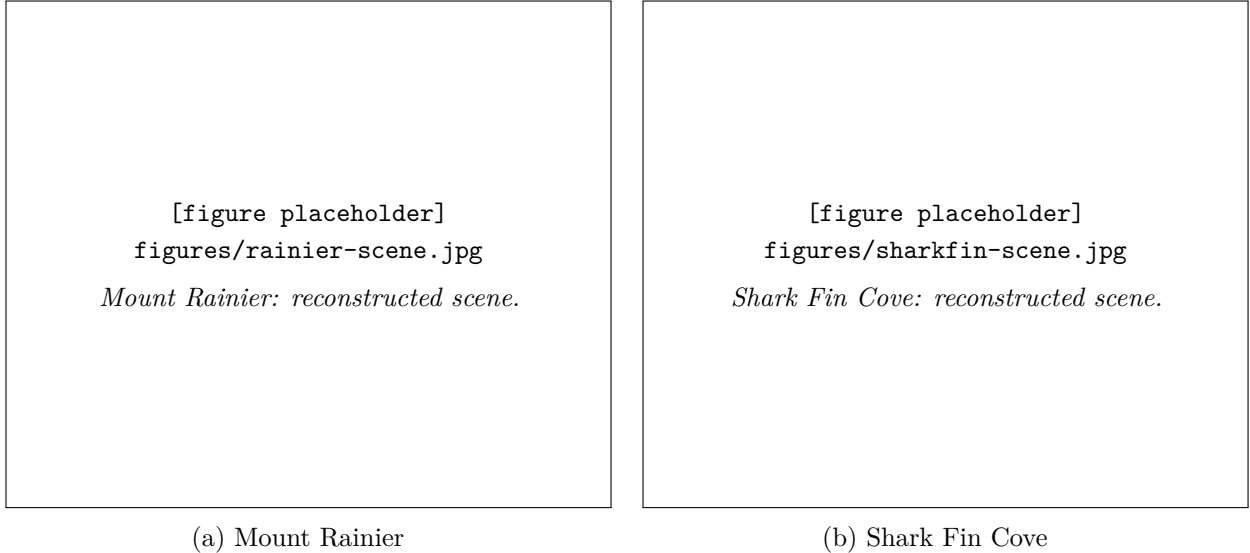


Figure 2: Reconstructed environments viewed from near the capture point.

Table 4: Scene composition per capture. *Relief added* is the elevation range introduced by terrain reconstruction beyond what the height field measured.

Capture	Meshes	Cards	Billboards	Relief added	Dominant limitation
Mount Rainier	128	9 303	33	35 m	Conifer reconstruction
Paris	27	4	56	78 m	Skyline read as landform
Shark Fin Cove	0	1	42	81 m	Detections are rock fragments
Iceland	0	45	39	52 m	As Shark Fin
Irises	0	10 738	0	—	Painting; ground cover only

11 Results and Discussion

11.1 Evaluation setup

We evaluate on five captures chosen to stress different assumptions: **Mount Rainier** (alpine meadow, dense vegetation, snow), **Paris** (flat urban river scene, architecture), **Shark Fin Cove** (coastal cliffs, open water, sea stacks), **Iceland** (barren volcanic terrain, human figures), and **Irises** (a Van Gogh painting — non-photographic input).

Ground truth 3D does not exist for any of them, so evaluation is necessarily diagnostic: we instrument the pipeline, measure intermediate quantities against what the scene demonstrably contains, and localise defects to specific stages.

Figure 2 shows two reconstructed environments viewed from near the capture point.

11.2 Quantitative summary

Table 4 summarises scene composition. The mesh/card/billboard split is the clearest single indicator of how well object reconstruction served each capture.

11.3 What works

Terrain geometry on landform scenes. Where the horizon genuinely is terrain, reconstruction behaves as intended. Measured cells are preserved exactly by construction, the harmonic solve produces a continuous surface, and erosion supplies drainage-consistent relief. On Mount Rainier the 35 m of added relief corresponds to real mountain that extends beyond depth range.

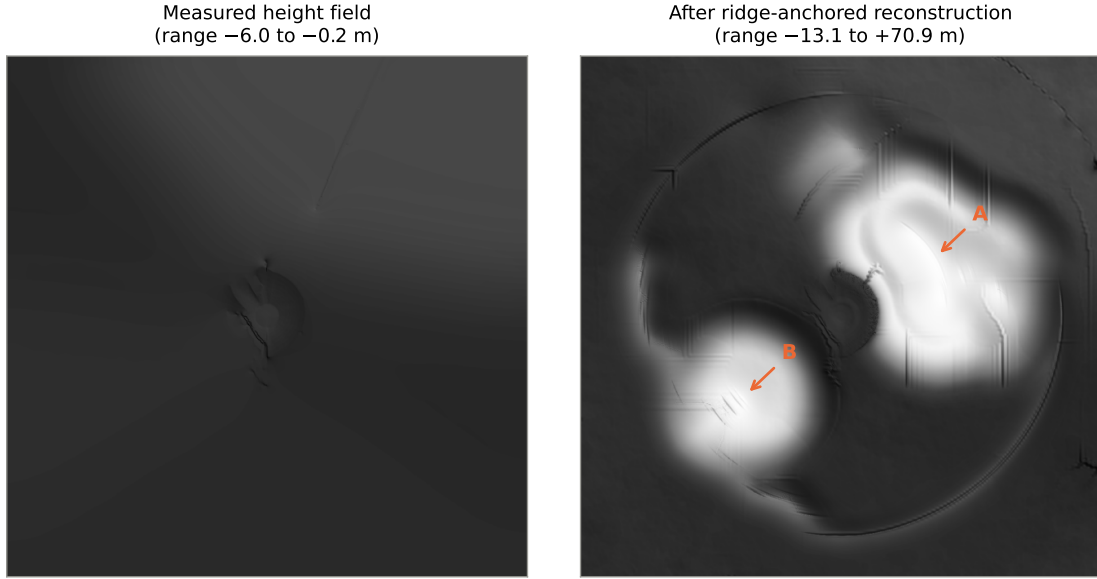


Figure 3: The Paris failure. **Left:** the measured height field, spanning -6.0 m to -0.2 m — correctly flat, and near-featureless under a shared shading scale. **Right:** after ridge-anchored reconstruction, spanning -13.1 m to 70.9 m . Hill A (5966 m^2) is the right-bank treeline; hill B (2674 m^2) is the cathedral. Both panels share one shading scale, so the left panel’s flatness is the measurement, not a rendering choice.

UV retention and fold repair. Retaining observed UVs measurably improved texture correspondence. The geometric fold detector identified 5 624 triangles (7.0%) on Mount Rainier bridging occlusion boundaries; collapsing them removed the concentric artefacts on the mountain face while leaving well-behaved triangles untouched. Detection converges in two passes.

Projected-size level of detail. Replacing distance-based LOD with angular size halved the scene’s triangle count ($4.47\text{ M} \rightarrow 2.48\text{ M}$) while retaining every prominent object, and removed the need for a per-class override table — ground cover falls below any sensible threshold on its own.

Ground-cover masking. Colour-vetoed grass masking correctly excludes snowfields that type identically to meadow. After ordering the veto to run after morphological closing, zero grass instances remain on cells the veto rejected.

11.4 Failure modes and their causes

The more instructive result is where the system fails, because in every case we were able to localise the failure to an *assumption* rather than a model. Section 11.5 reports what each corrective measure then achieved.

Skyline read as landform. Ridge anchoring assumes the sky boundary is the crest of ground. In Paris it is a roofline 40 m away. The measured height field spans -6.0 m to -0.2 m — flat, correctly — and reconstruction produced -13.1 m to 70.9 m : a 5966 m^2 hill on the right-bank treeline and a 2674 m^2 hill on the cathedral (fig. 3). The discriminator is what the silhouette is *made of*: the fraction of silhouette columns typing terrain is 0% for Paris against 87%, 98% and 100% for Iceland, Rainier and Shark Fin. Paris is not merely lowest; not one column of its horizon is a landform (fig. 4).

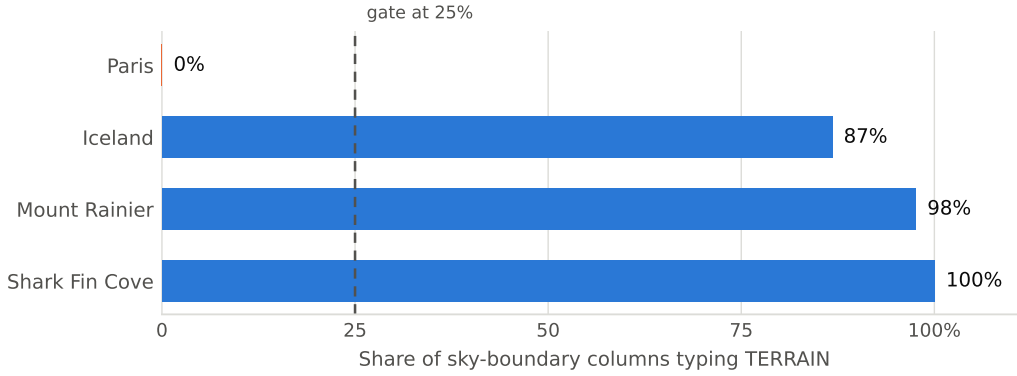


Figure 4: What each capture’s horizon is made of. The gate at 25 % separates Paris from every landform capture by a wide margin in both directions; it is a threshold on evidence the pipeline already computes, not a tuned constant.

Confident misclassification on texture. Shark Fin Cove produced 50 detections at median confidence 0.95, of which essentially none is an object. The stored captions are diagnostic: a rock typed *person* is captioned “a piece of pizza with bacon on it”; another typed *aircraft* is “a giraffe that is flying in the sky”. On captures with weak image signal the class is derived from the caption — 59% of Shark Fin crops and 91% of Rainier’s — and the captioner describes unparseable texture fluently and wrongly. These pass label verification, because a tight cutout fills its own box and an open-vocabulary detector will localise almost any label inside it.

The consequence for reconstruction is direct: SAM 3D given a flat rock fragment returns a flat sheet (normalised extents [1.00, 0.049, 0.425]), the shape gates correctly reject it, and every instance falls back to a billboard. Shark Fin’s zero meshes are the gates working, on garbage input.

The signal that separates them is the region map, because it is the only opinion in the chain that does not look at the crop. A per-crop plausibility veto — a class that cannot occupy the region its box sits in — removes 43% of Shark Fin’s detections with no false positives across the other captures.

Thin vegetation reconstruction. Conifers fail single-view reconstruction. Given a good 120×326 crop, SAM 3D returns a mesh with aspect 3.50:1 against detections at 0.12:1 — a $28\times$ disagreement — so 70 of 75 instances fall back to crossed cards. A thin conifer offers almost no volume cue from one view. Species-aware approaches [19] are the natural remedy.

Water surface following terrain. Water inherited the terrain height under each of its cells, which is correct only if the ground under a lake is flat. On Shark Fin one $11\,872\text{ m}^2$ body of sea spanned 73.5 m in elevation, with 21.7% of its vertices more than a metre above the median — the ocean draped up the cliffs.

Far-field texture resolution. Even with folds repaired, 67.8% of Shark Fin’s terrain area exceeds 0.5 m per texel against 20.9% for Rainier, because a clifftop camera places most of its surface near the horizon where equirectangular compression is worst (fig. 5).

11.5 Corrective measures and their measured effect

Each failure above was addressed by making its implicit premise testable from evidence already computed elsewhere in the pipeline, rather than by tuning a threshold or replacing a model. Table 5 reports the measured effect of each, evaluated by replaying the modified stage over saved pipeline state from all five captures. We report effects on *every* capture, not only the one that

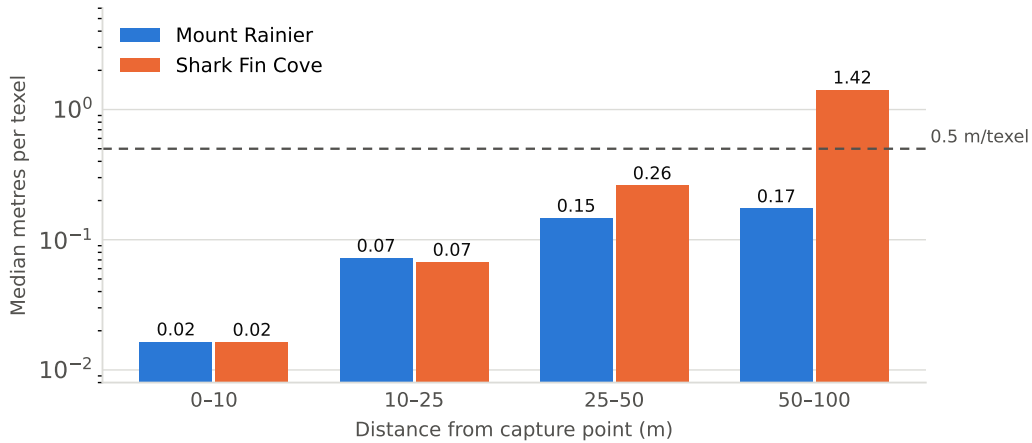


Figure 5: Texture resolution against distance, log scale. The two captures are indistinguishable within 25 m; beyond it Shark Fin degrades far faster, because a clifftop camera places most of its surface near the horizon where equirectangular compression is worst. Above roughly 0.5 m per texel the ground reads as smeared.

motivated the change, because the risk being managed is regression on scenes that were already correct.

Two of these deserve comment because the obvious version of each is measurably wrong.

Caption plausibility requires conjunction. Rejecting a detection because its caption mentions something implausible is appealing and unsafe. On Mount Rainier, 21 flower detections carry captions such as “a close up of a flower in a vase” — and 19 of those 21 are real, placed flowers. The captioner identifies the *subject* correctly and confabulates the *setting*. Conversely, rejecting on scene-tag support alone is also unsafe: Rainier’s tags name no tree, and it has 67 real ones. Only the conjunction of four conditions — no scene-tag support, implausible caption vocabulary, caption not naming the class, and class not vegetation — produced zero false positives. Notably, the caption–class agreement test is unusable as a veto (its synonym gaps delete real objects) but safe as an *exemption*, because a gap then merely retains a spurious detection rather than removing a real one.

Robust estimation must respect sampling density. Water levelling initially estimated each body’s surface from its mesh vertices. Because the mesh is Poisson-disc sampled with a near-camera density bias, 4309 of Shark Fin’s 11 366 water vertices lie within 20 m of the camera, and the resulting estimate described the cove at the viewer’s feet rather than the sea: -4.54 m against a shoreline at -0.99 m. Estimating over the uniform grid instead gives -1.13 m and a 0.14 m shoreline step. The estimator was correct; the sample weighting was not.

11.6 Strengths and weaknesses

Strengths. The decomposition is the system’s main asset. Because terrain, objects and ground cover are separate, each can be fixed without disturbing the others, and defects localise cleanly. Preserving measured data as hard constraints means the pipeline never silently overwrites evidence with inference. The instrumentation — every stage writing structured debug output — is what made the failure analysis above possible at all; several defects were invisible until the relevant quantity was recorded.

Weaknesses. Error compounds along a 40-stage chain, and a defect can surface many stages from its cause: object size compression originates in depth calibration and manifests in placement.

Table 5: Corrective measures, the premise each makes testable, and measured effect across captures. “FP” denotes false positives: correct content wrongly removed.

Measure	Premise made testable	Measured effect
Silhouette composition gate	“The sky boundary is the crest of ground”	Terrain-typed silhouette fraction: Paris 0%; Iceland 87%; Rainier 98%; Shark Fin 100%. Ridge anchoring suppressed on Paris only; other captures bit-identical.
Per-crop region plausibility	“This class can occupy the region its box sits in”	Removes 9/109 (Paris), 4/237 (Rainier), 12/49 (Shark Fin), 5/101 (Iceland). 0 FP; Rainier’s four are phantom aircraft inside the snowfield.
Caption-scene plausibility	“The caption describes something this scene can contain”	Removes a further 16 (Shark Fin), 16 (Iceland), 9 (Rainier), 2 (Paris). 0 FP. Requires four conjunctive conditions; each weaker variant deletes real content (see text).
Water surface leveling	“The ground beneath water is flat”	Shark Fin elevation spread 73.5 m $\rightarrow$ 0.00 m; resulting shoreline discontinuity 0.14 m. Paris (already near-level) 1.67 m $\rightarrow$ 0.55 m across 12 bodies.
Projected-size LOD	“Distance predicts resolvability”	Rainier meshed instances 545 $\rightarrow$ 296; triangles 4.47 M $\rightarrow$ 2.48 M; per-class override table eliminated.
Resolution-based asset selection	“The best-scoring crop is the best reconstruction input”	Source crop area increased 1.0–9.2 $\times$ per group; 205 of 471 instances had been rejected as degenerate reconstructions.
Geometric UV-fold repair	“Adjacent grid cells observed adjacent panorama rows”	Collapses 5624 triangles (Rainier, 7.0%), 2169 (Shark Fin), 1479 (Paris). Converges in two passes; face count unchanged.
Occluder-footprint inpainting	“What foreground removal left behind is ground”	Below-horizon pixels showing fabricated fill: 23.9% $\rightarrow$ 17.1%.

Scene-type assumptions are embedded implicitly, and the system has no notion of what *kind* of scene it is processing; the Paris terrain failure is exactly this. Reconstruction quality is bounded by single-view 3D, which is unreliable for thin structures. And the system has no way to know it is wrong: it produces the same confident output on a coastal cliff face typed as furniture as on a correctly reconstructed meadow.

Cost. End-to-end generation takes several hours on a single GPU, dominated by per-group 3D reconstruction (roughly 22 minutes for 18 groups) and video generation (around 15 minutes). This is offline content creation, not interactive authoring.

12 Conclusions and Future Work

12.1 What we learned

The most transferable lesson is that at this scale, **most defects are assumption failures rather than model failures**. Every significant problem we diagnosed took the same shape: a step valid for the scene category it was designed against, applied silently to a category where its premise does not hold. Ridge anchoring assumes the horizon is ground. Foreground removal assumes what

is close is an occluder. Water assumes the ground beneath it is flat. Distance-based LOD assumes objects are similarly sized. In each case the component worked exactly as specified and produced a bad scene.

This suggests that scene-type awareness should be explicit rather than emergent. The fixes that worked best were those that made a premise testable from evidence already present — measuring what the silhouette is made of, asking what region a crop sits in — rather than those that tuned a threshold.

A second lesson concerns **cross-checking between independent signals**. The strongest diagnostics came from disagreement between models that look at different evidence. A caption and a classifier that both read the same crop fail together; a region map segmenting the panorama fails independently, and that independence is what makes it useful.

Third, **instrumentation is a scientific instrument, not a convenience**. Defects we had looked at directly for weeks became obvious the moment the right quantity was logged.

12.2 Future work

Explicit scene classification. A first-class scene-type estimate (natural landform, urban, coastal, interior) consumed by every stage that currently embeds an implicit assumption would address the largest class of failure we observed.

Confidence propagation. Stages produce point estimates. Propagating uncertainty would let downstream stages weigh evidence rather than accept it, and would let the system report where it is guessing.

Better single-view reconstruction for thin structures. Vegetation is the dominant content of natural scenes and the weakest part of our object track; species-aware priors [19] are the obvious direction.

Joint refinement. The pipeline is strictly feed-forward. A closing optimisation that adjusted depth, geometry and placement jointly against photometric consistency could correct compounded error that no single stage can see.

Perceptual evaluation. Our evaluation is diagnostic. A user study measuring presence, scale perception and navigation comfort in the headset would test what the system is actually for.

12.3 Closing

Into Frame demonstrates that a single photograph can be turned into an environment a person can walk through, with terrain, objects, ground cover, water and lighting reconstructed as distinct physical components. The decomposition is what makes the result explorable rather than merely viewable, and it is also what made the system debuggable. The remaining failures are, encouragingly, not mysterious: each is a stated assumption meeting a scene that violates it, which is a tractable problem in a way that “the model produced a bad result” is not.

References

- [1] Danial Barazandeh and Gabriel Zachmann. Earthbender: An interactive system for stylistic heightmap generation using a guided diffusion model. In *Proceedings of the 2025 18th ACM SIGGRAPH Conference on Motion, Interaction, and Games*, MIG '25, New York, NY, USA, 2025. Association for Computing Machinery.

- [2] Katherine R Barnhart, Eric WH Hutton, Gregory E Tucker, Nicole M Gasparini, Erkan Istanbuluoglu, Daniel EJ Hobley, Nathan J Lyons, Margaux Mouchene, Sai Siddhartha Nudurupati, Jordan M Adams, et al. Landlab v2. 0: a software package for earth surface dynamics. *Earth Surface Dynamics*, 8(2):379–397, 2020.
- [3] Marcos V. Conde, Ui-Jin Choi, Maxime Burchi, and Radu Timofte. Swin2sr: Swinv2 transformer for compressed image super-resolution and restoration. In *Computer Vision – ECCV 2022 Workshops*, volume 13802 of *Lecture Notes in Computer Science*, pages 669–687. Springer, 2022.
- [4] Arnaud Emilien, Ulysse Vimont, Marie-Paule Cani, Pierre Poulin, and Bedrich Benes. Worldbrush: interactive example-based synthesis of procedural virtual worlds. *ACM Trans. Graph.*, 34(4), July 2015.
- [5] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, and Robin Rombach. Scaling rectified flow transformers for high-resolution image synthesis. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 12606–12633. PMLR, 2024.
- [6] Jean-David G enevaux, Eric Galin, Adrien Peytavie, Eric Gu erin, Cyril Briquet, Fran ois Grosbellet, and Bedrich Benes. Terrain Modelling from Feature Primitives. *Computer Graphics Forum*, 34(6):198–210, May 2015.
- [7] Charles J. Geyer and Jesper M oller. Simulation procedures and likelihood inference for spatial point processes. *Scandinavian Journal of Statistics*, 21(4):359–373, 1994.
- [8] Daniel EJ Hobley, Jordan M Adams, Sai Siddhartha Nudurupati, Eric WH Hutton, Nicole M Gasparini, Erkan Istanbuluoglu, and Gregory E Tucker. Creative computing with landlab: an open-source toolkit for building, coupling, and exploring two-dimensional numerical models of earth-surface dynamics. *Earth Surface Dynamics*, 5(1):21–46, 2017.
- [9] Xinyu Huang, Yi-Jie Huang, Youcai Zhang, Weiwei Tian, Rui Feng, Yuejie Zhang, Yanchun Xie, Yaqian Li, and Lei Zhang. Open-set image tagging with multi-grained text supervision. In *Proceedings of the 33rd ACM International Conference on Multimedia (MM)*, pages 4117–4126. ACM, 2025.
- [10] Xinyu Huang, Youcai Zhang, Jinyu Ma, Weiwei Tian, Rui Feng, Yuejie Zhang, Yaqian Li, Yandong Guo, and Lei Zhang. Tag2text: Guiding vision-language model via image tagging. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [11] Yukun Huang, Yanning Zhou, Jianan Wang, Kaiyi Huang, and Xihui Liu. DreamCube: 3D Panorama Generation via Multi-plane Synchronization. 2025.
- [12] Yukun Huang, Yanning Zhou, Jianan Wang, Kaiyi Huang, and Xihui Liu. DreamCube: RGB-D Panorama Generation via Multi-plane Synchronization. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 24922–24932, October 2025.
- [13] Zixuan Huang, Mark Boss, Aaryaman Vasishta, James M. Rehg, and Varun Jampani. SPAR3D: Stable point-aware reconstruction of 3d objects from single images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16860–16870, 2025.
- [14] T. Hurtut, P.-E. Landes, J. Thollot, Y. Gousseau, R. Drouillhet, and J.-F. Coeurjolly. Appearance-guided synthesis of element arrangements by example. In *Proceedings of the 7th International Symposium on Non-Photorealistic Animation and Rendering, NPAR ’09*, page 51–60, New York, NY, USA, 2009. Association for Computing Machinery.

- [15] Eric Hutton, Katy Barnhart, Dan Hobley, Greg Tucker, Sai Nudurupati, Jordan Adams, Nicole Gasparini, Charlie Shobe, Ronda Strauch, Jenny Knuth, Margaux Mouchene, Nathan Lyons, David Litwin, Rachel Glade, Giuseppe Cipolla, Amanda Manaster, Langston Abby, Kristen Thyng, and Francis Rengers. landlab, 4 2020.
- [16] Aryamaan Jain, James Gain, and Guillaume Cordonnier. Pixels2Peaks: Converting Terrain Images to Heightmaps. *ACM Transactions on Graphics*, 45(4), July 2026.
- [17] Derong Jin, Xiyi Chen, Ming C. Lin, and Ruohan Gao. Sonoworld: From one image to a 3d audio-visual scene. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.
- [18] Nikolai Kalischek, Michael Oechsle, Fabian Manhardt, Philipp Henzler, Konrad Schindler, and Federico Tombari. Cubediff: Repurposing diffusion-based image models for panorama generation. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- [19] Jae Joong Lee, Bosheng Li, Sara Beery, Jonathan Huang, Songlin Fei, Raymond A Yeh, and Bedrich Benes. Tree-d fusion: Simulation-ready tree dataset from single images with diffusion priors. In *European Conference on Computer Vision*, pages 439–460. Springer, 2024.
- [20] Junnan Li, Dongxu Li, Caiming Xiong, and Steven C. H. Hoi. BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *Proceedings of Machine Learning Research*, pages 12888–12900. PMLR, 2022.
- [21] Wenbo Li, Zhe Lin, Kun Zhou, Lu Qi, Yi Wang, and Jiaya Jia. Mat: Mask-aware transformer for large hole image inpainting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- [22] Ruofan Liang, Kai He, Zan Gojcic, Igor Gilitschenski, Sanja Fidler, Nandita Vijaykumar, and Zian Wang. Luxdit: Lighting estimation with video diffusion transformer. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025.
- [23] Lightricks. LTX-2: Open-weights synchronised video and audio generation. Open-weights model release, 2026. Model release with an accompanying technical report; no peer-reviewed publication. This work uses the LTX-2.3 22B checkpoint.
- [24] Haotong Lin, Sili Chen, Junhao Liew, Donny Y. Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth anything 3: Recovering the visual space from any views, 2025.
- [25] Xin Lin, Meixi Song, Dizhe Zhang, Wenxuan Lu, Haodong Li, Bo Du, Ming-Hsuan Yang, Truong Nguyen, and Lu Qi. Depth any panoramas: A foundation model for panoramic depth estimation. *arXiv*, 2025.
- [26] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *Computer Vision – ECCV 2024*, volume 15105 of *Lecture Notes in Computer Science*, pages 38–55. Springer, 2024.
- [27] Jundan Luo, Duygu Ceylan, Jae Shin Yoon, Nanxuan Zhao, Julien Philip, Anna Fröhstück, Wenbin Li, Christian Richardt, and Tuanfeng Y. Wang. IntrinsicDiffusion: Joint intrinsic layers from latent diffusion models. In *SIGGRAPH 2024 Conference Papers*, 2024.
- [28] Fabrice Neyret and Marie-Paule Cani. Pattern-Based Texturing Revisited. In *26th Annual Conference on Computer Graphics and interactive techniques (SIGGRAPH '99)*, pages 235–242, Los Angeles, United States, August 1999. ACM SIGGRAPH.

- [29] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mido Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024, 2024.
- [30] Avinash Paliwal, Xilong Zhou, Andrii Tsarov, and Nima Kalantari. Panodreamer: Optimization-based single image to 360° 3d scene with diffusion. In *Proceedings of the SIGGRAPH Asia 2025 Conference Papers*, 2025.
- [31] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763. PMLR, 2021.
- [32] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloé Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- [33] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10684–10695, June 2022.
- [34] Antoine Schnepf, Karim Kassab, Flavian Vasile, and Andrew Comport. SphericalDreamer: Generating Navigable Immersive 3D Worlds with Panorama Fusion. In *Forty-third International Conference on Machine Learning*, 2026.
- [35] Roman Suvorov, Elizaveta Logacheva, Anton Mashikhin, Anastasia Remizova, Arsenii Ashukha, Aleksei Silvestrov, Naejin Kong, Harshith Goka, Kiwoong Park, and Victor Lempitsky. Resolution-robust large mask inpainting with fourier convolutions. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 3172–3182, 2022.
- [36] SAM 3D Team, Xingyu Chen, Fu-Jen Chu, Pierre Gleize, Kevin J Liang, Alexander Sax, Hao Tang, Weiyao Wang, Michelle Guo, Thibaut Hardin, Xiang Li, Aohan Lin, Jiawei Liu, Ziqi Ma, Anushka Sagar, Bowen Song, Xiaodong Wang, Jianing Yang, Bowen Zhang, Piotr Dollár, Georgia Gkioxari, Matt Feiszli, and Jitendra Malik. Sam 3d: 3dfy anything in images. 2025.
- [37] Ziyi Wu, Daniel Watson, Andrea Tagliasacchi, David J. Fleet, Marcus A. Brubaker, and Saurabh Saxena. 360Anything: Geometry-free lifting of images and videos to 360°. *arXiv*, 2026.
- [38] Jianfeng Xiang, Xiaoxue Chen, Sicheng Xu, Ruicheng Wang, Zelong Lv, Yu Deng, Hongyuan Zhu, Yue Dong, Hao Zhao, Nicholas Jing Yuan, and Jiaolong Yang. Native and compact structured latents for 3d generation. *Tech report*, 2025.
- [39] Enze Xie, Wenhai Wang, Zhiding Yu, Anima Anandkumar, José M. Álvarez, and Ping Luo. Segformer: Simple and efficient design for semantic segmentation with transformers. In *Advances in Neural Information Processing Systems 34 (NeurIPS)*, pages 12077–12090, 2021.

- [40] Ziyang Xie. Worldgen: Generate any 3d scene in seconds. <https://github.com/ZiYang-xie/WorldGen>, 2025.
- [41] Shuai Yang, Jing Tan, Mengchen Zhang, Tong Wu, Gordon Wetzstein, Ziwei Liu, and Dahua Lin. Layerpano3d: Layered 3d panorama for hyper-immersive scene generation. In *ACM SIGGRAPH 2025 Conference Papers*. ACM, 2025.
- [42] Tao Yu, Runseng Feng, Ruoyu Feng, Jinming Liu, Xin Jin, Wenjun Zeng, and Zhibo Chen. Inpaint anything: Segment anything meets image inpainting. *arXiv preprint arXiv:2304.06790*, 2023.
- [43] Youcai Zhang, Xinyu Huang, Jinyu Ma, Zhaoyang Li, Zhaochuan Luo, Yanchun Xie, Yuzhuo Qin, Tong Luo, Yaqian Li, Shilong Liu, Yandong Guo, and Lei Zhang. Recognize anything: A strong image tagging model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 1724–1732, 2024.
- [44] Jixin Zhao, Zhouxia Wang, Peiqing Yang, and Shangchen Zhou. Precise object and effect removal with adaptive target-aware attention. In *CVPR*, 2026.
- [45] Peng Zheng, Dehong Gao, Deng-Ping Fan, Li Liu, Jorma Laaksonen, Wanli Ouyang, and Nicu Sebe. Bilateral reference for high-resolution dichotomous image segmentation. *CAAI Artificial Intelligence Research*, 3:9150038, 2024.